Sol 3 — Eyes

Camera, intrinsics and ArUco markers

Sol 3

https://github.com/KrithinP/vanguard-inductions-2026

Every link in this document opens the live page on GitHub.

PART 1

Your mission

Give the rover a camera and find markers in 3-D

SOL 3 — "Eyes"

```
╔═══════════════════════════════════════════════════════╗
║ SOL 031 · LANDING + 30 ║
║ SPROSCAPE · marker field ║
║ STATUS: NAVIGATING BY DEAD RECKONING ║
╠═══════════════════════════════════════════════════════╣
║ Give it eyes. Stop trusting the wheels. ║
╚═══════════════════════════════════════════════════════╝
```

Objective: give the rover a camera, and teach it to recognise where it is from what it sees. **Effort:** 6–8 hours.

Mission briefing

Your rover drove a square and didn't quite come home. That gap is dead reckoning failing: wheels slip, and small errors add up with nothing to correct them. Drive long enough and the rover's belief about its position becomes fiction.

The fix is to look at something you recognise. If you can see a landmark whose position you know, you can work out where *you* are — no matter how badly the wheels have been lying to you.

On a real mission those landmarks are **markers**: printed patterns placed at known points around the site. Spot one, measure where it is relative to you, and your position snaps back to the truth. The European Rover Challenge — the competition you'll be flying in 2027 — uses exactly this, with **ArUco markers** for localisation and path correction.

This Sol, you give the rover eyes.

A step up. Sol 1 handed you a working node to copy. From here the handbook gives you the API, the message formats and the traps — **you write the node.** That is the point: working from a description rather than an example is most of the job.

Tasks

1 · Add a camera

Mount a camera on the rover in your URDF and bridge its image topic into ROS.

Verify with `rqt_image_view` and in RViz. If you can see the world from the rover's point of view, the hard part is done.

2 · Understand an image message

A `sensor_msgs/msg/Image` is a flat array of bytes plus a description of how to interpret it — width, height, and an **encoding** like `rgb8` or `bgr8`.

Getting the encoding wrong gives you a picture with the red and blue channels swapped. It's a rite of passage. Recognise it fast.

3 · ROS images ↔ OpenCV

`cv_bridge` converts between the two. Write a node that subscribes to the camera, converts to grayscale, and publishes the result on a new topic. View both.

Small, but it proves the whole pipeline before you build anything real on it.

4 · Camera intrinsics

Look at `/camera_info`. Those numbers — focal length and optical centre — describe how the lens maps the 3-D world onto a flat sensor.

You need them for step 6, because working out *how far away* something is from a flat picture is impossible without knowing how the picture was made.

5 · Detect markers

> Note: **OpenCV 5 removed the old ArUco functions.** Almost every tutorial online uses `cv2.aruco.detectMarkers(...)`, which no longer exists. The handbook has the API that actually works. Read it before you search.

Generate some ArUco markers and write a node that finds them. Two parts:

a. Against the provided test images — this is the graded part. `markers/test_images/` holds six images with known answers. Run your detector over all six and report what it finds. It works the same on every machine, so nobody is advantaged by hardware.

One of them contains **no marker**. A detector that reports something there is worse than one that finds nothing — on a real rover it drives you into a rock.

b. Live in the Gazebo world — the demo. Place markers in your world and detect them from the rover's camera. Draw the outline and ID and publish it so you can watch it happen.

> If the marker renders as a **solid black square** in Gazebo, that's software rendering failing to load the texture, not your code. It's common in Docker. Say so in your log and lean on part (a). You are not marked down for it.

6 · Estimate marker pose

Using the intrinsics, work out **where each marker is relative to the camera** — distance and orientation, not just presence in the frame.

Publish it as a TF frame. Then look in RViz: the marker should appear in 3-D, in the right place, moving correctly as the rover drives.

That moment — a thing you detected in a flat image showing up correctly positioned in 3-D space — is the whole of robotic perception in miniature.

7 · Sanity-check yourself

Park the rover a measured distance from a marker. Compare your estimate to the truth. **Write both numbers in your `MISSION_LOG.md`.**

Be honest about the error. Everyone's is worse than they expect; the ones we're interested in are the people who measured it and can say *why*.

8 · Mission log

Deliverables

`src/sol3/`	camera setup, detection node, pose estimation
detector output	what your detector found on all six test images
screenshot	RViz with marker frames placed in 3-D
`MISSION_LOG.md`	updated, including measured vs. estimated distance
video	under 90 s — live detection with the rover moving

In the recording, **say out loud why camera intrinsics are needed to estimate distance.**

Checks

```
./tools/vanguard check
```

PART 2

Cameras

Image messages, encodings and cv_bridge

20 — Cameras in simulation

Time: 60 minutes. **By the end:** your rover has a camera, and you can see what it sees.

Mounting it

A camera is a link like any other, plus a Gazebo sensor. Add to your URDF:

```
<link name="camera_link">
<visual>
<geometry><box size="0.03 0.08 0.03"/></geometry>
<material name="lens"><color rgba="0.1 0.1 0.1 1"/></material>
</visual>
<inertial>
<mass value="0.1"/>
<inertia ixx="1e-4" ixy="0" ixz="0" iyy="1e-4" iyz="0" izz="1e-4"/>
</inertial>
</link>

<joint name="camera_joint" type="fixed">
<parent link="base_link"/>
<child link="camera_link"/>
<origin xyz="0.3 0 0.12" rpy="0 0 0"/>
</joint>

<gazebo reference="camera_link">
<sensor name="rover_camera" type="camera">
<update_rate>30</update_rate>
<always_on>true</always_on>
<topic>camera/image_raw</topic>
<camera>
<horizontal_fov>1.047</horizontal_fov>
<image><width>640</width><height>480</height><format>R8G8B8</format></image>
<clip><near>0.05</near><far>50.0</far></clip>
</camera>
</sensor>
</gazebo>
```

Note `type="fixed"` — the camera is bolted on, it doesn't move independently.

`horizontal_fov` is in **radians**. 1.047 ≈ 60°.

You also need the sensors system in your world file, or no sensor produces anything:

```
<plugin filename="gz-sim-sensors-system"
name="gz::sim::systems::Sensors">
<render_engine>ogre2</render_engine>
</plugin>
```

Under software rendering (Docker, most VMs) `ogre2` can be slow or unstable. If the simulator crashes on start, try `<render_engine>ogre</render_engine>`, and drop the camera to 320×240 at 10 Hz. A lower-resolution camera that runs beats a high-resolution one that doesn't.

Bridging it

Images and camera info both need bridging:

```
ros2 run ros_gz_bridge parameter_bridge \
 /camera/image_raw@sensor_msgs/msg/Image[gz.msgs.Image \
 /camera/camera_info@sensor_msgs/msg/CameraInfo[gz.msgs.CameraInfo
```

Both are `[` — Gazebo to ROS. You read from a camera; you don't write to it.

> Images are large. For high rates, `ros_gz_image`'s `image_bridge` is more efficient than `parameter_bridge`. At 640×480/30 Hz either is fine.

Looking at it

```
ros2 run rqt_image_view rqt_image_view
```

Pick `/camera/image_raw` from the dropdown. You should see the world from the rover's point of view. Drive it and watch the picture move.

In RViz: **Add → Image**, set the topic. Handy because you get the camera and the 3-D view side by side.

Sanity checks:

```
ros2 topic hz /camera/image_raw # should be near your update_rate
ros2 topic echo /camera/camera_info --once
```

What an image message actually is

```
ros2 interface show sensor_msgs/msg/Image
```

```
std_msgs/Header header # timestamp + which frame it was taken in
uint32 height # rows
uint32 width # columns
string encoding # what the bytes mean, e.g. "rgb8"
uint8 is_bigendian
uint32 step # bytes per row
uint8[] data # the pixels, one flat array
```

So an image is **a flat array of bytes plus instructions for interpreting it**. `data` has no structure of its own — `height`, `width`, `step` and `encoding` are what turn it back into a picture.

Encodings, and the classic bug

Encoding	Meaning
`rgb8`	3 bytes per pixel, red-green-blue
`bgr8`	3 bytes per pixel, **blue-green-red**
`mono8`	1 byte per pixel, grayscale
`32FC1`	one 32-bit float per pixel (depth, in metres)

OpenCV uses BGR. ROS commonly publishes RGB. Get it backwards and everything red looks blue. It's a rite of passage — the useful part is recognising it in one second instead of debugging your detector for an hour.

`cv_bridge`

`cv_bridge` converts between ROS `Image` messages and OpenCV arrays.

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class Grayscaler(Node):
 def __init__(self):
 super().__init__('grayscaler')
 self.bridge = CvBridge()
 self.sub = self.create_subscription(
 Image, '/camera/image_raw', self.on_image, 10)
 self.pub = self.create_publisher(Image, '/camera/image_gray', 10)

 def on_image(self, msg):
 # ROS message -> OpenCV array. 'bgr8' is what OpenCV expects.
 frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

 # Back to a ROS message. Note we do NOT use cv_bridge here — see below.
 # Keep the original header so the timestamp and frame_id survive; TF
 # needs them later.
 self.pub.publish(to_image_msg(gray, 'mono8', msg.header))

def main(args=None):
 rclpy.init(args=args)
 node = Grayscaler()
 try:
 rclpy.spin(node)
 except KeyboardInterrupt:
 pass
 finally:
 node.destroy_node()
 rclpy.shutdown()
```

Publishing an image back — and a trap

You'd expect `cv_bridge` to convert both ways. It doesn't, on our stack:

Direction	Function	Works?
ROS → OpenCV	`imgmsg_to_cv2`	[required] yes
OpenCV → ROS	`cv2_to_imgmsg`	**`KeyError: 16`**

`cv_bridge` ships with ROS 2 Jazzy and was built for OpenCV 4. Our OpenCV is 5, which renumbered its internal type constants, so `cv2_to_imgmsg` can't map them and raises `KeyError`. Nothing you did is wrong.

The fix is five lines, and it's worth writing once because it shows you exactly what an `Image` message is:

```
from sensor_msgs.msg import Image

def to_image_msg(array, encoding, header):
 """numpy array -> sensor_msgs/Image, without cv_bridge.

 Works because an Image message is just the raw bytes plus the numbers
 needed to interpret them: height, width, encoding and step.
 """
 msg = Image()
 msg.header = header
 msg.height = array.shape[0]
 msg.width = array.shape[1]
 msg.encoding = encoding
 msg.is_bigendian = 0
 channels = 1 if array.ndim == 2 else array.shape[2]
 msg.step = array.shape[1] * channels * array.itemsize
 msg.data = array.tobytes()
 return msg
```

`step` is bytes per row. Get it wrong and your image comes out skewed diagonally — which is a memorable way to learn what `step` means.

Two habits worth forming now:

1. **Always pass `desired_encoding`.** It converts for you, so your code doesn't depend on what the publisher happened to choose.
2. **Copy the header onto anything you publish.** Losing the timestamp and `frame_id` breaks TF lookups downstream, and the failure appears far from the cause.

Once it runs, view both topics in `rqt_image_view` side by side. Small, but it proves your whole pipeline works before you build anything real on it.

If it went wrong

No image topic at all — the sensors system plugin is missing from the world, or the sensor isn't attached to a real link. Run `gz topic -l` to see what Gazebo has.

Topic exists but nothing arrives — the bridge direction is wrong. It must be `[` (Gazebo→ROS).

Simulator crashes when the camera loads — rendering. Try `ogre` instead of `ogre2`, and lower the resolution.

`ImportError: No module named cv_bridge` — `sudo apt install ros-jazzy-cv-bridge`

A wall of text about "a module compiled using NumPy 1.x cannot be run in NumPy 2.x" — your NumPy got upgraded past what `cv_bridge` was built against. Fix and re-check:

```
pip install --user --break-system-packages "numpy<2"
python3 -c "import numpy; print(numpy.__version__)" # must start with 1.
```

`./tools/vanguard doctor` checks this too.

`KeyError: 16` from `cv2_to_imgmsg` — expected. Use `to_image_msg` above.

Colours inverted — RGB vs BGR. See above.

Very low frame rate — expected under software rendering. Drop to 320×240 and 10 Hz; nothing in this mission needs more.

Next: `21-intrinsics.md`.

PART 3

Why a flat image can give you a distance

Camera intrinsics

21 — Camera intrinsics

Time: 30 minutes. **By the end:** you understand why a flat picture can tell you a distance.

The question

A camera flattens the 3-D world onto a 2-D sensor. That throws away depth — a small rock nearby and a large rock far away can produce **identical** pixels.

So how can you possibly get a distance out of one image?

You can't — unless you know how big the thing is. If you know the marker is exactly 15 cm across, and you know how the lens maps the world onto pixels, then its apparent size tells you its distance. Intrinsics are the second half of that.

The camera matrix

```
ros2 topic echo /camera/camera_info --once
```

```
height: 480
width: 640
k: [554.25, 0.0, 320.5,
 0.0, 554.25, 240.5,
 0.0, 0.0, 1.0]
d: [0.0, 0.0, 0.0, 0.0, 0.0]
```

`k` is the **camera matrix**, usually written K:

```
⎡ fx 0 cx ⎤
K = ⎢ 0 fy cy ⎢
⎣ 0 0 1 ⎦
```

- **fx, fy — focal length in pixels.** How strongly the lens magnifies. Bigger = narrower field of view, things look larger.
- **cx, cy — the optical centre**, where the lens axis meets the sensor. Usually near the image centre (here 320.5, 240.5 for a 640×480 image — as expected).

`d` holds **distortion** coefficients, for lenses that bend straight lines. Our simulated camera is perfect, so they're all zero. A real camera needs calibration to find them.

The relationship, in one line

For an object of real size **S** appearing **w** pixels wide at distance **Z**:

```
w = fx · S / Z which rearranges to Z = fx · S / w
```

Check it against the numbers above: a 15 cm marker filling 300 pixels with fx = 600 gives Z = 600 × 0.15 / 300 = **0.30 m**. That's the whole idea. Everything else is doing it properly in 3-D and for corners rather than widths.

This is why intrinsics are not optional. Without fx you have a ratio and no scale — you can say "twice as far as before" but never "1.4 metres".

Getting them in code

```
from sensor_msgs.msg import CameraInfo
import numpy as np

class Detector(Node):
 def __init__(self):
 ...
 self.K = None
 self.dist = None
 self.create_subscription(
 CameraInfo, '/camera/camera_info', self.on_info, 10)

 def on_info(self, msg):
 self.K = np.array(msg.k, dtype=np.float64).reshape(3, 3)
 self.dist = np.array(msg.d, dtype=np.float64)
```

Then guard your image callback:

```
 def on_image(self, msg):
 if self.K is None:
 return # no intrinsics yet — can't estimate anything
```

`camera_info` is often published less frequently than images, and sometimes only once. **Store it when it arrives; don't assume it's there.** Not guarding this is a guaranteed startup crash.

Frame conventions — the one that will catch you

OpenCV and ROS do not agree on which way a camera points.

	x	y	z
OpenCV (optical)	right	down	**forward**
ROS (body, REP-103)	forward	left	up

So a translation of `[0, 0, 2.0]` from OpenCV means *2 m in front of the camera*. Publish that straight into a ROS frame that follows the body convention and your marker appears 2 m **above** the rover.

Two ways to handle it:

1. **Convention (recommended).** Name your camera's optical frame `camera_optical_frame` and add a fixed joint rotating it from `camera_link`: `xml <joint name="camera_optical_joint" type="fixed"> <parent link="camera_link"/> <child link="camera_optical_frame"/> <origin xyz="0 0 0" rpy="-1.5708 0 -1.5708"/> </joint>` Publish detections in `camera_optical_frame` and TF handles the rest. This is what ROS expects, and why you see `_optical_frame` everywhere in real stacks.
2. **By hand.** Swap the axes yourself. Works, but you'll get it wrong once and it'll take an hour to find.

> If your marker shows up in RViz at roughly the right *distance* but in a completely wrong *direction* — this is why. It's the most common bug here.

Next: `22-aruco.md`.

PART 4

Markers

Detection and pose estimation

22 — ArUco markers

Time: 90 minutes. **By the end:** your rover recognises markers and knows where they are in 3-D.

What a fiducial is

A **fiducial marker** is a pattern designed to be easy for a computer to find: high contrast, a thick black border, and an internal grid encoding an ID number.

ArUco markers are square, black-and-white, and come from predefined *dictionaries*. `DICT_4X4_50` means a 4×4 internal grid with 50 distinct IDs.

Why they matter to us: the **European Rover Challenge** uses ArUco markers for localisation and path correction in the autonomous navigation task, and on the maintenance panel. Detecting them is not an exercise — it's the job.

> Smaller dictionaries (4×4) are easier to detect at distance but tolerate less error. `DICT_4X4_50` is a sensible default. Don't use a 7×7 dictionary and then wonder why nothing is detected from 3 m away.

Note: Read this before you copy any tutorial

OpenCV 5 removed the old ArUco API. Every blog post, StackOverflow answer and YouTube tutorial written before 2024 uses functions that **no longer exist**:

Old (gone — will raise `AttributeError`)	Current
`cv2.aruco.Dictionary_get(...)`	`cv2.aruco.getPredefinedDictionary(...)`
`cv2.aruco.detectMarkers(img, dict, ...)`	`detector.detectMarkers(img)` on an `ArucoDetector`
`cv2.aruco.drawMarker(...)`	`cv2.aruco.generateImageMarker(...)`
`cv2.aruco.estimatePoseSingleMarkers(...)`	`cv2.solvePnP(...)`

If you get `module 'cv2.aruco' has no attribute 'detectMarkers'`, you have copied a pre-2024 tutorial. That error is *the* signal. Everything below is written against the API that actually ships with our container.

Check what you have:

```
python3 -c "import cv2; print(cv2.__version__); print(hasattr(cv2.aruco,'ArucoDetector'))"
```

```
5.0.0
True
```

Making markers

```
import cv2
import numpy as np

aruco = cv2.aruco
dictionary = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)

for marker_id in (0, 1, 2, 3):
 img = aruco.generateImageMarker(dictionary, marker_id, 400)
 # a white quiet zone round the edge is REQUIRED for detection
 canvas = np.full((500, 500), 255, np.uint8)
 canvas[50:450, 50:450] = img
 cv2.imwrite(f'marker_{marker_id}.png', canvas)
```

> **The white border is not decoration.** The detector finds markers by looking for a black quad against a lighter background. A marker flush to the edge of a texture, or on a dark wall, often won't be found at all.

Putting them in the world

> **Note: Textures may not render, and that is not your fault**
>
> Gazebo renders PBR textures through the `ogre2` engine. **With software rendering — which is what you get in Docker, and in most VMs — the texture often doesn't load and the marker renders as a solid black square.** Gazebo logs no error. Detection then finds nothing, correctly, because there is no pattern there.
>
> We verified this: on a headless container the marker face renders pure black. With a real GPU it generally works.
>
> **So the graded part of this mission does not depend on it.** We ship `markers/test_images/` — six images with known answers — and your detector is scored on those. Getting markers rendering in Gazebo is the demo, not the grade. If you see a black square, you have not done anything wrong; run against the images and say so in your log.

Add a thin box to your `.sdf` world and apply the marker PNG as a texture:

```
<model name="marker_0">
<static>true</static>
<pose>3 0 0.5 0 0 0</pose>
<link name="link">
<visual name="visual">
<geometry><box><size>0.02 0.15 0.15</size></box></geometry>
<material>
<pbr><metal>
<albedo_map>materials/textures/marker_0.png</albedo_map>
</metal></pbr>
</material>
</visual>
<collision name="collision">
<geometry><box><size>0.02 0.15 0.15</size></box></geometry>
</collision>
</link>
</model>
```

The box is thin in **x** (0.02 m) and 0.15 m square in y–z, so its printed face already points back down the −x axis at a rover approaching from the origin. **No yaw rotation is needed** — adding one turns the face side-on and you'll see a thin black sliver instead of a marker. (We made exactly that mistake while writing this.)

Check in the GUI that you can actually see the square face, before blaming your detector. Two different problems look similar from the outside:

What you see	Cause
a thin vertical black line	marker is edge-on — remove the rotation
a solid black square	the texture didn't load — see the warning above
the pattern	you're fine

Keep the physical size (0.15 m here) written down. You need it for pose estimation and it must match reality exactly.

Detecting

From here on you write the code. Sol 1 handed you a working node to copy; this one gives you the API and the traps, and you assemble it. That step up is deliberate.

The calls you need

```
aruco = cv2.aruco
dictionary = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)
detector = aruco.ArucoDetector(dictionary, aruco.DetectorParameters())

corners, ids, rejected = detector.detectMarkers(gray_image)
```

- Build the dictionary and detector **once, in `__init__`**. Rebuilding them per frame is wasteful and a common beginner smell.
- `detectMarkers` wants a **grayscale** image. Convert with `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)`.
- `corners` is a list of arrays, one per marker, each `(1, 4, 2)` — four (x, y) image points, in order: top-left, top-right, bottom-right, bottom-left. **That order matters in the next section.**

- `ids` is an `(N, 1)` array, or **`None`** when nothing is found — not an empty list. `if ids is not None:` — `len(ids)` raises.

The node you're building

Structure it yourself, but it needs to:

- subscribe to the image topic **and** `/camera/camera_info`
- store the intrinsics when they arrive, and **return early from the image callback until they have** — camera_info is often published once, and later than the first frame
- detect, and log which IDs it saw
- draw the result with `cv2.aruco.drawDetectedMarkers(frame, corners, ids)` and publish it, so you can watch it live in `rqt_image_view`

Get detection visible before attempting pose. **Everything after this is debugged by looking at that image**, so it is worth the twenty minutes.

Pose estimation

`estimatePoseSingleMarkers` no longer exists. Use `cv2.solvePnP` directly, which is what it called internally anyway.

The idea: you know where the marker's four corners are in *its own* coordinates — a flat square of known size, centred on itself. You have measured where those corners landed in the image. `solvePnP` solves for the rotation and translation that explains the difference.

What you have to work out

1. **The object points.** Four 3-D points describing the marker's corners in its own frame, `z = 0` because it's flat, sized by your real marker. They must be in the **same order** `detectMarkers` returns: top-left, top-right, bottom-right, bottom-left. Get the order wrong and the pose is a plausible- looking lie.
2. **The image points.** `corners[i]` reshaped to `(4, 2)` and cast to `float32`.
3. **The call.** `cv2.solvePnP(object_points, image_points, K, dist, flags=...)` returns `(success, rvec, tvec)`. Use **`cv2.SOLVEPNP_IPPE_SQUARE`** — it is built for exactly this case and is far more stable than the default.

Reading the answer

`tvec` is the marker's position **in the camera's optical frame**, in metres: x right, y down, **z forward**. So `tvec[2]` is how far in front of the camera it is — that's the number to check first against a tape measure.

`rvec` is a compact axis-angle rotation. `cv2.Rodrigues(rvec)` turns it into a 3×3 matrix when you need one.

Check it visually before you trust it:

```
cv2.drawFrameAxes(frame, K, dist, rvec, tvec, marker_size / 2)
```

The axes should sit flat **on** the marker. Floating, tilted or scaled wrongly means your object points are wrong — almost always the order or the size.

Publishing it as a TF frame

A pose that only exists inside your node isn't much use. Broadcast it so the rest of ROS — and RViz — can see it.

You'll need a `tf2_ros.TransformBroadcaster`, and to fill a `geometry_msgs/msg/TransformStamped` with the translation from `tvec` and a **quaternion** converted from `rvec`. `scipy.spatial.transform.Rotation` will do the matrix→quaternion step in one line (note it returns **x, y, z, w** — ROS wants the same order, but check, because half the libraries in this field disagree). Doing the conversion by hand is about ten lines and a good exercise.

Two details that decide whether this works:

- **Use the image's timestamp**, `msg.header.stamp` — **not** `now()`. The pose describes where the marker was when the picture was taken. Using "now" makes TF lookups fail or return quietly wrong answers while the rover is moving, and the error surfaces nowhere near the cause.
- **`frame_id` must be your camera's optical frame**, not `camera_link`. See `21-intrinsics.md`. Publishing into the body frame puts the marker in a completely wrong direction while the distance still looks right — which is exactly how you waste an evening.

Then in RViz: Fixed Frame `odom`, add **TF**, and drive around. **The marker frame should stay still in the world while the rover moves.** If it slides along with the rover, your transform is being published in the wrong frame.

Scoring your detector — the provided test images

`markers/test_images/` holds six images with known answers in `markers/expected.json`:

Image	Contains
`01_single_close`	one marker, straight on
`02_single_far`	small and slightly blurred
`03_rotated`	rotated in the image plane
`04_two_markers`	two at once
`05_off_centre`	near the edge, dim lighting
`06_none`	**nothing — must return no detections**

Run your detector over all six and print what it finds. **This is the part we grade**, because it works identically on every machine regardless of graphics.

`06_none` is not a throwaway. A detector that reports a marker when there isn't one is worse than useless on a rover — it will send it driving at a rock. Make sure yours handles `ids is None` rather than crashing or inventing something.

Measure your error

Park the rover a measured distance from a marker (read the true value from the world file and `/odom`). Compare with `tvec[2]`.

Record both numbers in your `MISSION_LOG.md`. Then try it at 1 m, 3 m and 5 m — the error will not be constant, and understanding why is more valuable than a small number.

If it went wrong

`module 'cv2.aruco' has no attribute 'detectMarkers'` You copied a pre-2024 tutorial. See the warning at the top.

`KeyError: 16` from `cv2_to_imgmsg` Expected on our stack — `cv_bridge` was built for OpenCV 4, we run OpenCV 5. Use `to_image_msg` from `20-cameras.md`. Reading *into* OpenCV with `imgmsg_to_cv2` is fine; only the way back out is affected.

`A module compiled using NumPy 1.x cannot be run in NumPy 2.x`

```
pip install --user --break-system-packages "numpy<2"
```

Nothing is ever detected In order: is the marker facing the camera? Is there a white border? Is the image actually arriving (`ros2 topic hz`)? Is the resolution too low, or the marker too far? View `/markers/debug` — if the picture looks fine to you and still nothing is found, try moving much closer to confirm the pipeline works at all.

Detected, but the ID is wrong Wrong dictionary. The one you generate with must be the one you detect with.

Distance is consistently out by a constant factor `MARKER_SIZE` doesn't match the world. Distance scales linearly with it.

Pose flips or jitters wildly between frames Normal for a square marker viewed nearly head-on — there are two solutions that project almost identically. `SOLVEPNP_IPPE_SQUARE` helps; viewing at a slight angle helps more.

Marker appears at the right distance but the wrong direction in RViz Optical vs body frame convention. `21`.

`Lookup would require extrapolation into the past` You used `now()` instead of the image timestamp, or `use_sim_time` is inconsistent between nodes.

Back to the Sol 3 sheet.